

C++ Interview Guide

Junior / Mid C++ հարցազրույցի պատրաստության ուղեցույց

Այս ուղեցույցը ընդգրկում է C++ հարցազրույցների ամենահաճախ հանդիպող թեմաները՝ Constructor, Destructor, Virtual Destructor, Reference-ներ, `new`, temporary objects, `std::string_view`, compiler optimization-ներ և SSO: Յուրաքանչյուր բաժնում կա բացատրություն, կոդի օրինակ, real-life analogy և պատրաստի interview պատասխան:

1. Constructor

Ի՞նչ է Constructor-ը

Constructor-ը class-ի հատուկ ֆունկցիա է, որը **ավտոմատ կանչվում է, երբ օբյեկտ է ստեղծվում**: Նրա հիմնական նպատակը օբյեկտի սկզբնական (initial) վիճակը կարգավորելն է:

```
class Person {  
public:  
    string name;  
  
    Person() {  
        name = "Unknown";  
    }  
};  
  
int main() {  
    Person p;  
}
```

Այստեղ `Person p;` ստեղծելիս constructor-ը ավտոմատ կանչվում է:

Constructor-ի հիմնական հատկությունները

1. Անունը class-ի անվան հետ նույնն է

```
class Person {  
public:  
    Person() {  
    }  
};
```

2. Return type չունի — Սխալ կլինի գրել `void Person() {}`: Constructor-ը return type չի ունենում:

3. Ավտոմատ է կանչվում — `Person p;` տողի ժամանակ constructor-ը ինքն է աշխատում:

Default Constructor

Parameter չունեցող constructor-ը:

```
class Car {
public:
    Car() {
        cout << "Car created";
    }
};

// Car c;  -> Output: Car created
```

Parameterized Constructor

Երբ constructor-ը parameter է ընդունում:

```
class Person {
public:
    string name;

    Person(string n) {
        name = n;
    }
};

int main() {
    Person p("Arman");    // name-ը միանգամից ստանում է արժեք
}
```

Constructor Overloading

Մի class-ը կարող է ունենալ մի քանի constructor՝ տարբեր parameter-ներով:

```

class Person {
public:
    Person() {
        cout << "Default\n";
    }
    Person(string name) {
        cout << name << endl;
    }
    Person(string name, int age) {
        cout << name << ", " << age << endl;
    }
};

// Person p1;
// Person p2("Arman");
// Person p3("Arman", 25);

```

Առանց overloading-ի ստիպված կլինեիր միշտ նույն քանակի parameter ուղարկել:

Initialization List

Junior մակարդակում հաճախ հարցնում են: Սա ավելի արդյունավետ է, քան constructor-ի մարմնի մեջ assignment անելը:

```

class Person {
public:
    string name;

    Person(string n) : name(n) {
    }
};

```

Real-life analogy

Պատկերացրու մեքենա ես գնում գործարանից: Constructor-ը այն գործընթացն է, որը մեքենան պատրաստում է օգտագործման համար՝ շարժիչ է դնում, անիվներ է դնում, գույն է տալիս: Այսինքն՝ օբյեկտի ստեղծման պահին նրա «կարգավորումն» է անում:

Interview Answer (կարճ տարբերակ)

A constructor is a special member function of a class that is automatically called when an object is created. Its purpose is to initialize the object's data members and prepare the object for use. A constructor has the same name as the class and does not have a return type.

Կարևոր follow-up հարցեր

Constructor can be private? — Այո:

```
class Singleton {  
private:  
    Singleton() {}  
};
```

Can constructor be overloaded? — Այո: `Person(); Person(string); Person(string, int);`

Can constructor be virtual? — Ոչ: Շատ սիրած interview հարց է:

No, constructors cannot be virtual because the object does not yet exist when the constructor is called.

2. Private Constructor & Singleton

Private Constructor

```
class Test {  
private:  
    Test() {}  
};  
  
int main() {  
    Test t;    // ERROR: 'Test::Test()' is private  
}
```

Class-ից դուրս ոչ ոք չի կարող object ստեղծել, քանի որ main-ը իրավունք չունի կանչելու private constructor-ը:

Singleton Pattern

Singleton-ը նշանակում է՝ ծրագրում այդ class-ից պետք է լինի ընդամենը **1 object**:

```
class Singleton {  
private:  
    Singleton() {}  
  
public:  
    static Singleton& getInstance() {  
        static Singleton instance;  
        return instance;  
    }  
};  
  
// Singleton& s1 = Singleton::getInstance();  
// Singleton& s2 = Singleton::getInstance();  
// &s1 == &s2    (սույն object-ն է)
```

`static Singleton instance;` object-ը ստեղծվում է միայն մեկ անգամ: Հաջորդ կանչերի ժամանակ նոր object չի ստեղծվում, միևնույն instance-ն է վերադարձվում:

Real Life Example

Պատկերացրու Database Connection ունենալ: Չես ուզում ամեն function-ի համար նոր connection բացվի, դա ռեսուրսների վատնում է: Փոխարենը՝ `Database::getInstance()` — բոլորը նույն connection-ն են օգտագործում:

Interview Follow-up

Ինչու constructor-ը private ենք դարձնում Singleton-ում

To prevent users from creating objects directly and ensure only one instance of the class exists.

Կարելի է object ստեղծել private constructor ունեցող class-ից — Դրսից՝ ոչ: Class-ի ներսից՝ այո:

```
class Test {
private:
    Test() {}
public:
    static Test create() {
        return Test(); // աշխատում է, class-ը սեփական private անդամներին հասանելիու
    }
};
```

Overloading vs Overriding

Շատ interview-ներում խառնում են այս երկուսը:

Overloading — նույն անուն, տարբեր parameters (compile-time):

```
Person();
Person(string);
Person(string, int);
```

Overriding — inheritance-ի թեմա է (run-time):

```
class Animal {  
public:  
    virtual void speak() { cout << "Animal"; }  
};  
  
class Dog : public Animal {  
public:  
    void speak() override { cout << "Dog"; }  
};
```

3. Destructor

Ի՞նչ է Destructor-ը

Destructor-ը class-ի հատուկ member function է, որը **ավտոմատ կանչվում է, երբ object-ը ոչնչացվում է (destroyed)**: Եթե constructor-ը object-ը «ստեղծում» է, destructor-ը object-ը «մաքրում» է:

```
class Person {
public:
    Person() {
        cout << "Constructor called\n";
    }
    ~Person() {
        cout << "Destructor called\n";
    }
};

int main() {
    Person p;
}

// Output:
// Constructor called
// Destructor called
```

Նկատի՞ր `~` նշանը՝ `~ClassName() { }`:

Երբ է կանչվում Destructor-ը

Case 1. Stack Object — Destructor-ը կանչվում է scope-ից դուրս գալու պահին:

```
{
    Person p;
} // հենց փակագիծը փակվեց -> Destructor called
```

Case 2. Heap Object — Destructor-ը ավտոմատ չի կանչվի, պետք է `delete` գրես:

```
Person* p = new Person();
delete p; // միայն այս դեպքում destructor-ը կաշխատի
```

Ինչի համար է պետք Destructor-ը

Հիմնական նպատակը resource cleanup-ն է՝ memory ազատել, file փակել, database connection փակել, socket փակել:

```
class Test {  
public:  
    int* data;  
  
    Test() {  
        data = new int[100];  
    }  
    ~Test() {  
        delete[] data;    // object-ը ոչնչանալիս հիշողությունը կազատվի  
    }  
};
```

Առանց destructor-ի՝ հիշողություն ենք վերցնում, բայց չենք վերադարձնում — **memory leak**:

Real Life Example

Պատկերացրո՛ւ սենյակ ես վարձել: Constructor-ը՝ մտար սենյակ, միացրիր լույսը: Destructor-ը՝ անջատեցիր լույսը, փակեցիր դուռը, դուրս եկար: Այսինքն cleanup է անում:

Destructor-ի սահմանափակումները

Parameter կարող է ունենալ — Ոչ: `~Person(int x) {}` սխալ է:

Return type կարող է ունենալ — Ոչ: `void ~Person() {}` սխալ է:

Destructor Overloading կա՞մ — Ոչ: Միայն մեկ destructor կարող է լինել:

No. A class can have only one destructor.

Constructor vs Destructor

Constructor	Destructor
Object creation	Object destruction
Կարող է parameter ունենալ	Parameter չունի
Կարող է overloaded լինել	Overloading չունի

Destructor Call Order (LIFO)

```
int main() {  
    Test a;  
    Test b;  
}  
// Constructor a  
// Constructor b  
// Destructor b    <- վերջինը ստեղծված object-ը առաջինն է ոչնչացվում  
// Destructor a
```

Սա կոչվում է **LIFO (Last In, First Out)**:

Interview պատասխաններ

What is a destructor?

A destructor is a special member function that is automatically called when an object is destroyed. It is used to release resources and perform cleanup.

When is it called?

For stack objects, it is called automatically when the object goes out of scope. For heap objects, it is called when delete is used.

Can a destructor have parameters? — No. Can a destructor be overloaded? — No.

4. Virtual Destructor

Object-ի կառուցվածքը inheritance-ի ժամանակ

Derived object-ը կազմված է երկու մասից՝ Base մասը և Derived մասը:

Ստեղծման ժամանակ՝ `Base Constructor` -> `Derived Constructor`: Ոչնչացման ժամանակ հակառակ հերթականությամբ՝ `Derived Destructor` -> `Base Destructor`:

Խնդիրը polymorphism-ի ժամանակ

```
class Base {
public:
    ~Base() { cout << "Base Destructor\n"; }
};

class Derived : public Base {
public:
    ~Derived() { cout << "Derived Destructor\n"; }
};

Base* ptr = new Derived();
delete ptr;
// Output: Base Destructor    <- ԽՆԴԻՐ, Derived destructor-ը չկանչվեց
```

Compiler-ը նայում է pointer-ի type-ին (`Base*`): Քանի որ destructor-ը virtual չէ, compile time-ում որոշում է կանչել `Base::~~Base()`, և `Derived::~~Derived()` չի կանչվում:

Ինչու է դա վտանգավոր

Եթե Derived-ը memory է պահում`

```

class Derived : public Base {
    int* data;
public:
    Derived() { data = new int[100]; }
    ~Derived() { delete[] data; cout << "Derived Destructor\n"; }
};

Base* ptr = new Derived();
delete ptr;    // delete[] data չի կատարվում -> MEMORY LEAK

```

Լուծումը

Base destructor-ը դարձնում ենք virtual:

```

class Base {
public:
    virtual ~Base() { cout << "Base Destructor\n"; }
};

Base* ptr = new Derived();
delete ptr;
// Output:
// Derived Destructor
// Base Destructor    <- հիմա ճիշտ է

```

Ինչ է անում virtual keyword-ը

`virtual ~Base()` ասում է compiler-ին.

Destructor-ը ընտրվի runtime-ում՝ ըստ object-ի հրահան type-ի, ոչ թե pointer-ի type-ի:

Սա նույն գաղափարն է, ինչ virtual function-ը՝ **runtime polymorphism**:

Երբ է virtual destructor պետք

If a class is intended to be used as a base class and objects may be deleted through a base class pointer, the destructor should be virtual.

Երբ virtual destructor **պարտադիր չէ** — եթե inheritance ընդհանրապես չկա, կամ երբեք base pointer-ով delete չես անում (`Derived d;` միայն stack-ի վրա):

Interview-ի կարճ պատասխան

What is a virtual destructor?

A virtual destructor ensures that when an object is deleted through a base class pointer, both the derived class destructor and the base class destructor are called correctly.

Why is it important?

Without a virtual destructor, deleting a derived object through a base pointer can cause resource leaks because the derived destructor may not be executed.

5. Reference-ներ և Temporary Objects

Non-const lvalue reference չի ընդունում temporary

```
void Foo(std::string& s) { }
```

```
Foo("Arman");    // COMPILE ERROR
```

"Arman" -ից compiler-ը ստեղծում է temporary `std::string`, իսկ non-const lvalue reference-ը (`std::string&`) չի կարող bind անել temporary-ին: Պատճառը՝ `std::string&`-ը ենթադրում է, որ ֆունկցիան կարող է փոփոխել օբյեկտը, իսկ temporary-ն մի քանի տող հետո կոչնչանա:

const lvalue reference-ն ընդունում է

```
void Foo(const std::string& s) { }
```

```
Foo("Arman");    // OK  
// std::string temp("Arman"); Foo(temp);
```

Reference-ը կապվում է temporary-ին, և temporary-ի lifetime-ը երկարացվում է մինչև կանչի ավարտը:

rvalue reference-ն էլ ընդունում է

```
void Foo(std::string&& s) { }
```

```
Foo("Arman");    // OK, &&-ը rvalue reference է, հենց temporary-ների համար է
```

Ամփոփ աղյուսակ

```
std::string&        // միայն lvalue  
const std::string&  // lvalue + temporary  
std::string&&       // միայն temporary (rvalue)
```

```
std::string name = "Arman";

Foo1(name);           // std::string&      OK
Foo1("Arman");        //                  ERROR

Foo2(name);           // const std::string& OK
Foo2("Arman");        //                  OK

Foo3(name);           // std::string&&     ERROR
Foo3("Arman");        //                  OK
```

Այստեղից են սկսվում `move semantics`, `perfect forwarding`, `universal references` թեմաները:

6. new int vs new int()

```
int* a = new int;           // uninitialized (աղբ արժեք) -> undefined behavior
int* b = new int();         // value initialization -> *b == 0
int* c = new int{};        // C++11, նույնպես *c == 0
```

`new int` -ը memory է allocate անում, բայց արժեքը չի initialize անում — մեջը կարող է աղբ լինել: `new int()` -ը value initialization է անում, արդյունքը 0 է:

7. Temporary Objects-ի կյանքը

Temporary object-ը compiler-ի կողմից կարճ ժամանակով ստեղծված օբյեկտ է:

```
std::string s = "Arman";  
// "Arman" -> string literal (const char[6]),  
// compiler-ը ստեղծում է temporary std::string ու դրանով initialize է անում s-ը
```

Երբ ֆունկցիային փոխանցում ենք "Arman" (by value)

```
void Foo(std::string s) { }  
Foo("Arman");
```

Քայլերը՝ "Arman" -> `const char[6]` -> compiler-ը կանչում է `std::string temp("Arman")` -> temporary-ն փոխանցվում է պարամետրին (սովորաբար move constructor-ով):

const std::string& — copy չկա

```
void Foo(const std::string& s) { }  
Foo("Arman");
```

Temporary std::string ստեղծվում է, reference-ը կապվում է դրան, copy չի լինում, lifetime-ը երկարացվում է մինչև ֆունկցիայի ավարտ:

std::string_view — ամենաօպտիմալը

```
void Foo(std::string_view sv) { }  
Foo("Arman");
```

`string_view` -ը պահում է միայն **pointer + length**: Ոչ allocation, ոչ copy, ոչ move — ընդհանրապես `std::string` չի ստեղծվում:

String փոխանցելու գործնական խորհուրդ

```
void Foo(std::string s);           // վատ, կարող է copy անել  
void Foo(const std::string& s);    // ավելի լավ, copy չկա  
void Foo(std::string_view s);      // շատ դեպքերում լավագույնը (եթե միայն կարդում ես)
```


8. Compiler Optimizations & SSO

RVO (Return Value Optimization)

```
std::string GetName() {  
    return std::string("Arman");  
}
```

Ժամանակակից compiler-ները `std::string`-ը անմիջապես ստեղծում են վերջնական տեղում՝ **No Copy, No Move**:

NRVO (Named Return Value Optimization)

```
std::string GetName() {  
    std::string name = "Arman";  
    return name;    // հաճախ compiler-ը այստեղ էլ copy/move չի անում  
}
```

SSO (Small String Optimization)

```
std::string s = "Arman";
```

Քանի որ `string`-ը փոքր է, heap allocation ընդհանրապես չի լինում — տառերը պահվում են հենց object-ի ներսում: Այսինքն փոքր `string`-ները շատ արագ են, heap չեն գնում:

Ինտերվյուի համար հիշիր

Թեմա	Հիմնական միտք
<code>new int</code>	Uninitialized
<code>new int()</code>	Initialized to 0
<code>const std::string&</code>	No copy, temporary lifetime extension
<code>std::string_view</code>	pointer + length, no allocation/copy/move
Compiler optimization	RVO, NRVO, Move semantics, Copy elision
<code>std::string</code>	Հաճախ SSO, փոքր <code>string</code> -ները heap չեն գնում

Ամփոփ Cheat Sheet

Constructor — Special member function called automatically when an object is created.

Constructor Overloading — Multiple constructors with different parameter lists.

Private Constructor — Prevents direct object creation from outside the class.

Singleton — Design pattern that ensures only one instance of a class exists.

Destructor — Special member function called automatically when an object is destroyed; used for cleanup.

Virtual Destructor — Ensures derived + base destructors both run when deleting through a base pointer.

Reference rules — `T&` only lvalue, `const T&` lvalue+temporary, `T&&` only rvalue.

new int / new int() — uninitialized vs value-initialized (0).

string_view — pointer + length, zero allocation for read-only access.

RVO / NRVO / SSO — key optimizations that eliminate copies and heap allocations.